

In Java, **static and generics interact in an important way** because static members belong to the **class**, while generic type parameters such as T belong to a **particular generic type/instance context**.

The key rule is:

A static member cannot directly use the class's type parameter.

Let's understand this from the ground up.

1. What is a generic class?

A generic class is a class that declares a **type parameter**.

```
class Box<T> {  
  
    private T value;  
  
    public void set(T value) {  
        this.value = value;  
    }  
  
    public T get() {  
        return value;  
    }  
}
```

Here:

- Box = generic class
- T = type parameter
- Box<String> = parameterized type
- Box<Integer> = another parameterized type

Example:

```
Box<String> b1 = new Box<>();  
b1.set("Hello");
```

```
Box<Integer> b2 = new Box<>();  
b2.set(100);
```

Conceptually:

```
Box<String>  
T → String
```

```
Box<Integer>
  T → Integer
```

Therefore, T can be different for different parameterized uses.

2. What does non-static mean?

A **non-static member** belongs to an **object/instance**.

```
class Box<T> {
    T value;

    void set(T value) {
        this.value = value;
    }

    T get() {
        return value;
    }
}
```

Each object has its own value.

```
Box<String> b1 = new Box<>();
Box<Integer> b2 = new Box<>();
```

Conceptually:

```
b1
└─ T = String
   └─ value = "Hello"

b2
└─ T = Integer
   └─ value = 100
```

Therefore, a non-static member **can use T**.

```
class Box<T> {
    T value;          // VALID

    void set(T value) { // VALID
        this.value = value;
    }

    T get() {         // VALID
```

```
    return value;
}
}
```

3. What does static mean?

A static member belongs to the **class itself**, rather than to individual objects.

```
class Box<T> {
    static int count;
}
```

There is only one count associated with the class, conceptually.

The problem is:

```
Box<String>
Box<Integer>
Box<Double>
```

All of these use the same runtime class:

Box

So which T should a static member use?

```
class Box<T> {
    static T value; // ✗Compilation error
}
```

Java cannot make this meaningful because T is an **instance/type-parameter context**, while static belongs to the class.

4. The fundamental rule

Member	Can directly use class type parameter T?
Instance field	✓ Yes
Instance method parameter	✓ Yes
Instance method return type	✓ Yes

Instance method local variable	✓ Yes
Static field	✗ No
Static method parameter	✗ No
Static method return type	✗ No
Static method local variable	✗ No
Static nested class	✗ Not directly

The important distinction is:

T
↓
belongs to generic class's type context

static
↓
belongs to class-level context

These two contexts cannot directly be mixed.

5. Example: static field with generic type

This is illegal:

```
class Box<T> {
    static T value;
}
```

The compiler reports an error similar to:

non-static type variable T cannot be referenced from a static context

Why?

Suppose Java allowed:

```
Box<String> stringBox = new Box<>();
Box<Integer> integerBox = new Box<>();
```

What would this mean?

Box<T>.value

Should it be:

String value

or:

Integer value

There is only one static field for the class, so the answer is ambiguous.

6. Instance field works

This is completely valid:

```
class Box<T> {  
    T value;  
}
```

Usage:

```
Box<String> stringBox = new Box<>();  
stringBox.value = "Hello";
```

```
Box<Integer> integerBox = new Box<>();  
integerBox.value = 100;
```

Each object has its own type-specific value.

7. Static method cannot directly use T

This is also illegal:

```
class Box<T> {  
    static T create() {  
        return null;  
    }  
}
```

Why?

Because:

```
static create()
```

belongs to Box itself, not to:

```
Box<String>  
Box<Integer>  
Box<Double>
```

Therefore, there is no particular T available.

8. But a static method CAN declare its own generic type

This is one of the most important concepts.

Although a static method cannot use the **class's T**, it can declare a **method-level type parameter**.

```
class Box<T> {  
  
    static <T> Box<T> create(T value) {  
        Box<T> box = new Box<>();  
        box.value = value;  
        return box;  
    }  
  
    T value;  
}
```

Here there are actually **two different Ts**.

```
class Box<T> {  
    ↑  
    class T
```

and:

```
static <T> Box<T> create(T value)  
    ↑  
    method T
```

They are independent type parameters.

9. Better example using different names

To avoid confusion:

```
class Box<T> {  
    T value;  
  
    static <U> Box<U> create(U value) {  
        Box<U> box = new Box<>();  
        box.value = value;  
        return box;  
    }  
}
```

Now:

T → class-level type parameter

U → method-level type parameter

Usage:

```
Box<String> b1 = Box.create("Hello");  
Box<Integer> b2 = Box.create(100);
```

Conceptually:

```
Box.create("Hello")
```

↓

U = String

```
Box.create(100)
```

↓

U = Integer

The static method does not depend on the class's T.

10. Static generic method vs generic class

These are two different mechanisms.

Generic class

```
class Box<T> {
```

```
    T value;
```

```
T get() {  
    return value;  
}
```

Here:

T belongs to Box

Generic static method

```
class Utility {  
  
    static <T> T identity(T value) {  
        return value;  
    }  
}
```

Here:

T belongs to identity()

They are independent.

11. Can a static method use a generic class?

Yes.

For example:

```
class Box<T> {  
  
    T value;  
  
    static <U> Box<U> create(U value) {  
        Box<U> box = new Box<>();  
        box.value = value;  
        return box;  
    }  
}
```

This is valid because the static method introduces its **own** type parameter U.

12. Static method with bounded generic type

You can also use bounds.

```
class Utility {  
  
    static <T extends Number> double square(T value) {  
        return value.doubleValue() * value.doubleValue();  
    }  
}
```

Usage:

```
System.out.println(Utility.square(10));  
System.out.println(Utility.square(10.5));
```

Here:

T extends Number

means T must be Number or a subclass.

13. Static generic field — how to do it correctly

This is illegal:

```
class Box<T> {  
  
    static T value; // ✗  
}
```

But this is legal:

```
class Box<T> {  
  
    static Object value;  
}
```

However, this loses type safety.

A better approach is often a **static generic method**:

```
class Box<T> {  
  
    static <U> U process(U value) {  
        return value;  
    }  
}
```

```
}  
}
```

14. Static field that is itself generic

Java does not have a "generic static variable" in the sense of:

```
static T value;    // ✗
```

But you can create a generic object as a static field:

```
class Example {  
    static Box<String> box = new Box<>();  
}
```

This is legal because `Box<String>` itself is a concrete parameterized type.

```
class Box<T> {  
    T value;  
}
```

Then:

```
class Example {  
    static Box<String> box = new Box<>();  
}
```

Here the static field has a specific type:

```
Box<String>
```

It does **not** depend on `Example<T>`.

15. Generic class with static constant

A very common pattern is:

```
class Box<T> {  
    static final int MAX_SIZE = 100;  
}
```

This is perfectly valid.

Why?

Because MAX_SIZE doesn't depend on T.

The problem isn't that static and generic classes cannot coexist.

The problem is specifically:

A static member cannot directly depend on the generic class's type parameter.

Therefore:

```
static int count;           // ✓  
static final String NAME = ""; // ✓  
static T value;             // ✗
```

16. Static method using non-generic information

This is also valid:

```
class Box<T> {  
    static int count = 0;  
  
    static void printCount() {  
        System.out.println(count);  
    }  
}
```

Neither member needs T.

17. Static method cannot access instance member directly

This is related to the same principle.

```
class Box<T> {  
    T value;  
  
    static void print() {  
        System.out.println(value); // ✗  
    }  
}
```

Why?

Because value belongs to an object.

A static method has no implicit object reference (this).

You can make it work by supplying an object:

```
class Box<T> {  
    T value;  
  
    static <U> void print(Box<U> box) {  
        System.out.println(box.value);  
    }  
}
```

Usage:

```
Box<String> box = new Box<>();  
box.value = "Hello";
```

```
Box.print(box);
```

18. Why this also cannot be used in static context

Consider:

```
class Box<T> {  
    T value;  
  
    static void test() {  
        System.out.println(this.value); // ✗  
    }  
}
```

this represents the current object.

But a static method has no current object.

Therefore:

```
static method  
↓  
no this  
↓  
cannot directly access instance members
```

And because the class's T is associated with the generic instance context, the static method cannot directly use T either.

19. Static nested class and generics

A static nested class is another important case.

```
class Outer<T> {  
    static class Inner {  
    }  
}
```

Inner cannot directly use Outer's T.

For example:

```
class Outer<T> {  
    static class Inner {  
        T value; // ✖  
    }  
}
```

This is illegal.

Why?

Because Inner is static and does not have an associated Outer<T> instance/type parameter context.

20. But the static nested class can be generic itself

This is valid:

```
class Outer<T> {  
    static class Inner<U> {  
        U value;  
        void set(U value) {  
            this.value = value;  
        }  
    }  
}
```

```
}  
}
```

Usage:

```
Outer.Inner<String> obj = new Outer.Inner<>();
```

```
obj.set("Hello");
```

Here:

```
Outer<T>
```

↓

not available directly to Inner

```
Inner<U>
```

↓

has its own U

21. Static nested class inside generic class

Consider:

```
class Outer<T> {  
  
    class NonStaticInner {  
        T value;    // ✓  
    }  
  
    static class StaticInner {  
        // T value; // ✗  
    }  
}
```

The difference is:

Nested class	Can directly use outer T?
Non-static inner class	✓
Static nested class	✗

Example:

```
class Outer<T> {  
  
    class Inner {
```

```
    T value;
}
}
```

works because the non-static inner class is associated with an instance of Outer<T>.

But:

```
class Outer<T> {
    static class Inner {
        T value; // ✗
    }
}
```

doesn't.

22. Non-static generic method

A non-static method can use the class's T.

```
class Box<T> {
    T value;

    T getValue() {
        return value;
    }
}
```

It can also introduce another type parameter:

```
class Box<T> {
    <U> U convert(U value) {
        return value;
    }
}
```

Here:

T → class-level type parameter
U → method-level type parameter

23. Static generic method

A static method must introduce its own type parameter if it needs one.

```
class Utility {  
    static <T> T identity(T value) {  
        return value;  
    }  
}
```

Usage:

```
String s = Utility.identity("Hello");
```

```
Integer i = Utility.identity(100);
```

The compiler infers:

```
identity("Hello")
```

↓

T = String

```
identity(100)
```

↓

T = Integer

24. Very important distinction: same name does not mean same T

Consider:

```
class Test<T> {  
    T instanceValue;  
  
    static <T> void method(T value) {  
        System.out.println(value);  
    }  
}
```

These are **not the same T**.

```
class Test<T>
```

↑

Class T

```
static <T> method(T value)
```

↑

Method T

The method-level T shadows the class-level T.

It would be clearer to write:

```
class Test<T> {  
  
    T instanceValue;  
  
    static <U> void method(U value) {  
        System.out.println(value);  
    }  
}
```

25. Static generic method inside generic class

A very useful example:

```
class Pair<K, V> {  
  
    K key;  
    V value;  
  
    static <X, Y> Pair<X, Y> of(X key, Y value) {  
        Pair<X, Y> pair = new Pair<>();  
        pair.key = key;  
        pair.value = value;  
        return pair;  
    }  
}
```

Usage:

```
Pair<String, Integer> pair =  
    Pair.of("Age", 30);
```

Here:

```
Pair<K,V>  
    K and V → class type parameters  
  
of(X,Y)  
    X and Y → method type parameters
```

The static method doesn't use K or V.

Instead, it creates its own generic context:

$X \rightarrow \text{String}$
 $Y \rightarrow \text{Integer}$

26. Static members shared across all parameterizations

Another important concept is **type erasure**.

Consider:

```
class Box<T> {  
    static int count = 0;  
  
    T value;  
}
```

Then:

```
Box<String> a = new Box<>();  
Box<Integer> b = new Box<>();
```

There isn't conceptually a separate:

```
Box<String>.count  
Box<Integer>.count
```

There is one:

```
Box.count
```

Therefore:

```
Box<String>.count = 10; // ✗syntax  
Box<Integer>.count = 20; // ✗syntax
```

You access it as:

```
Box.count = 10;
```

This demonstrates why a static field cannot be T.

27. Example showing the problem clearly

Imagine Java allowed:

```
class Box<T> {  
    static T value;  
}
```

Then:

```
Box<String>.value = "Hello";  
Box<Integer>.value = 100;
```

There would have to be two different static fields:

```
Box<String>  
    static value → String
```

```
Box<Integer>  
    static value → Integer
```

But Java's generic parameterization does not create separate runtime classes for these types.

Instead, generics primarily provide compile-time type safety through type erasure.

Therefore Java prohibits this design.

28. Generic static collection

This is illegal:

```
class Registry<T> {  
    static List<T> items = new ArrayList<>(); // ✗  
}
```

But this is valid:

```
class Registry<T> {  
    static List<String> items = new ArrayList<>();  
}
```

Because String is a concrete type.

Or:

```
class Registry<T> {
```

```
static List<?> items = new ArrayList<>();
}
```

Or:

```
class Registry<T> {

    static <U> List<U> createList() {
        return new ArrayList<>();
    }
}
```

29. Generic static method returning a collection

Example:

```
class CollectionUtil {

    static <T> List<T> create(T... values) {
        return Arrays.asList(values);
    }
}
```

Usage:

```
List<String> names =
    CollectionUtil.create("A", "B", "C");
```

```
List<Integer> numbers =
    CollectionUtil.create(10, 20, 30);
```

The method's T is independently inferred.

30. Static vs non-static with generics — complete comparison

Feature	Non-static	Static
Belongs to	Object	Class
Has this	✓ Yes	✗ No
Can directly use class T	✓ Yes	✗ No
Can access instance fields directly	✓ Yes	✗ No

Can access instance methods directly	✓ Yes	✗ No
Can access static fields	✓ Yes	✓ Yes
Can access static methods	✓ Yes	✓ Yes
Can declare its own <U>	✓ Yes	✓ Yes
Depends on particular object	Usually yes	No
Can be overridden	Instance methods can be overridden	Static methods are hidden, not overridden
Can directly depend on Box<T>'s T	✓	✗

31. The three different generic contexts

This is perhaps the most useful way to understand the entire topic.

A. Class-level generic

```
class Box<T> {
}
```

T belongs to the class.

B. Instance method-level generic

```
class Box<T> {
    <U> void method(U value) {
    }
}
```

T belongs to the class.

U belongs to the method.

C. Static method-level generic

```
class Box<T> {
    static <U> void method(U value) {
    }
}
```

```
}
```

U belongs to the static method.

T is unavailable directly inside the static method.

32. A complete example

```
class Box<T> {  
  
    // Class-level generic type  
    private T value;  
  
    // Static member that doesn't depend on T  
    private static int count;  
  
    // Instance method using class T  
    public void set(T value) {  
        this.value = value;  
    }  
  
    // Instance method using class T  
    public T get() {  
        return value;  
    }  
  
    // Static method with its own generic type U  
    public static <U> Box<U> create(U value) {  
  
        Box<U> box = new Box<>();  
  
        box.set(value);  
  
        count++;  
  
        return box;  
    }  
  
    // Static method without generic type  
    public static int getCount() {  
        return count;  
    }  
}
```

Usage:

```
Box<String> b1 = Box.create("Hello");
```

```
Box<Integer> b2 = Box.create(100);
```

```
System.out.println(b1.get());
```

```
System.out.println(b2.get());
```

```
System.out.println(Box.getCount());
```

Output:

Hello

100

2

Notice what happened:

```
Box<T>
├── T → instance/class generic context
├── value → instance member
└── count → static member

create()
└── U → method-level generic context
```

33. Common mistakes

Mistake 1

```
class Test<T> {
    static T data;    // ✗
}
```

Correct:

```
class Test<T> {
    T data;           // ✓
}
```

or:

```
class Test<T> {
```

```
static <U> U method(U data) {  
    return data;  
}  
}
```

Mistake 2

```
class Test<T> {  
  
    static T get() {    // ✖  
        return null;  
    }  
}
```

Correct:

```
class Test<T> {  
  
    static <U> U get() {  
        return null;  
    }  
}
```

Mistake 3

Thinking this:

```
static <T> void method(T value)
```

uses the class's T.

It doesn't.

For clarity:

```
class Test<T> {  
  
    static <U> void method(U value) {  
    }  
}
```

is better.

34. Rule to remember

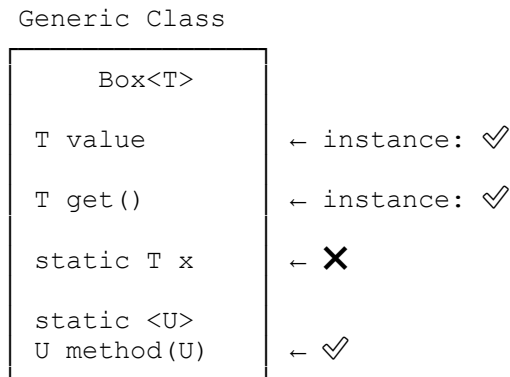
You can memorize this single rule:

A class's generic type parameter belongs to the generic class's instance/type context, so a static member cannot directly refer to it. If a static method needs generics, declare a method-level type parameter.

In short:

```
class Box<T>
  |
  | available to instance members
static
  |
  | cannot directly use class T
static <U>
  |
  | creates its own generic type parameter
```

Final mental model



The **most important distinction** is therefore not simply "static vs non-static", but **class-level type parameters vs method-level type parameters**. A static method can absolutely be generic; it just cannot directly use the **generic type parameter declared by its enclosing class**.

Upper and Lower Bounds in Java Generics

In Java Generics, **bounds** restrict the types that can be used with a type parameter or wildcard.

The two important concepts are:

- **Upper bound** → extends
- **Lower bound** → super

The simplest way to remember them is:

Upper bound: "This type must be this type or a subtype."

Lower bound: "This type must be this type or a supertype."

1. What is a bound?

A **bound** is a restriction placed on a generic type.

Without a bound:

```
class Box<T> {  
    T value;  
}
```

T can represent almost any reference type:

```
Box<String>  
Box<Integer>  
Box<Employee>  
Box<Number>
```

With a bound:

```
class NumberBox<T extends Number> {  
    T value;  
}
```

Now T must be Number or a subclass of Number.

Valid:

```
NumberBox<Integer>  
NumberBox<Double>  
NumberBox<Long>
```

Invalid:

NumberBox<String> // ✕

2. Upper bound

An **upper bound** specifies the **maximum/general type** that a type parameter can be.

Syntax:

<T extends SomeType>

It means:

T must be SomeType or a subtype of SomeType

For example:

<T extends Number>

means:

T = Number

T = Integer

T = Double

T = Long

T = Float

...

but not:

T = String

T = Object

3. Upper bound with a generic class

```
class Box<T extends Number> {
```

```
    private T value;
```

```
    public void set(T value) {  
        this.value = value;  
    }
```

```
    public T get() {  
        return value;  
    }  
}
```

Now:

```
Box<Integer> b1 = new Box<>();  
Box<Double> b2 = new Box<>();  
Box<Long> b3 = new Box<>();
```

are valid.

But:

```
Box<String> b4 = new Box<>();
```

is invalid.

Why?

Because:

```
String  
↓  
Object
```

but:

```
String
```

is not a subtype of:

```
Number
```

4. Why upper bounds are useful

The biggest advantage is that Java knows what operations are available on the bounded type.

Consider:

```
static <T> double square(T value) {  
    return value.doubleValue() * value.doubleValue();  
}
```

This doesn't compile because Java doesn't know whether T has doubleValue().

But:

```
static <T extends Number> double square(T value) {  
    return value.doubleValue() * value.doubleValue();  
}
```

works.

Why?

Because the compiler knows:

T extends Number

↓

T is Number or a subclass of Number

↓

Number.doubleValue() exists

Example:

```
System.out.println(square(5));  
System.out.println(square(5.5));  
System.out.println(square(10L));
```

5. Upper bound with interfaces

extends can also be used with interfaces.

```
interface Printable {  
    void print();  
}
```

Then:

```
class Utility {  
  
    static <T extends Printable> void display(T obj) {  
        obj.print();  
    }  
}
```

Any type passed to display() must implement Printable.

For example:

```
class Employee implements Printable {  
  
    @Override  
    public void print() {  
        System.out.println("Employee");  
    }  
}
```

Now:

```
Employee e = new Employee();
```

```
Utility.display(e); // ✓
```

But:

```
String s = "Hello";
```

```
Utility.display(s); // ✗
```

because String does not implement Printable.

6. Multiple upper bounds

Java allows multiple bounds.

Syntax:

```
<T extends ClassName & Interface1 & Interface2>
```

For example:

```
class Employee
    implements Comparable<Employee>, Serializable {

    @Override
    public int compareTo(Employee other) {
        return 0;
    }
}
```

A generic method can require both:

```
static <T extends Employee & Serializable> void process(T employee) {
}
```

The general structure is:

```
<T extends Class & Interface1 & Interface2>
```

Important rule:

If there is a class bound, it must come first.

Valid:

`<T extends Number & Serializable>`

Invalid:

`<T extends Serializable & Number>`

7. Upper-bounded wildcard

Upper bounds also occur with **wildcards**.

Syntax:

`? extends Type`

For example:

`List<? extends Number>`

This means:

A list of some unknown type that is Number or a subclass of Number.

It could be:

`List<Integer>`

`List<Double>`

`List<Long>`

`List<Number>`

8. Important difference: `<T extends Number>` vs `<? extends Number>`

These look similar but have different purposes.

Type parameter

`<T extends Number>`

This **declares a type variable**.

Example:

```
static <T extends Number> void process(T value) {  
}
```

Wildcard

`<? extends Number>`

This represents an **unknown type**.

Example:

`List<? extends Number> list;`

So:

Syntax	Meaning
<code><T extends Number></code>	Declare a type parameter T bounded by Number
<code><? extends Number></code>	Unknown type that is Number or a subtype

9. What can you do with `List<? extends Number>`?

Suppose:

`List<Integer> integers = new ArrayList<>();`

`List<? extends Number> numbers = integers;`

You can read values as Number:

`Number n = numbers.get(0);`

This is safe because whatever the actual list type is, it is some subtype of Number.

But this is not allowed:

```
numbers.add(10);           // ✗
numbers.add(20.5);         // ✗
```

Why?

Because Java doesn't know what the actual type is.

It could be:

`List<Integer>`

or:

List<Double>

If Java allowed:

```
numbers.add(20.5);
```

and the actual list were:

List<Integer>

the list would become type-unsafe.

10. Upper-bound wildcard = good for reading

A useful rule is:

? extends T → Producer → mainly read from it.

Example:

```
static double sum(List<? extends Number> numbers) {  
    double total = 0;  
    for (Number n : numbers) {  
        total += n.doubleValue();  
    }  
    return total;  
}
```

This accepts:

```
List<Integer>  
List<Double>  
List<Long>  
List<Float>
```

Example:

```
List<Integer> integers =  
    Arrays.asList(10, 20, 30);
```

```
List<Double> doubles =  
    Arrays.asList(10.5, 20.5);
```

```
System.out.println(sum(integers));  
System.out.println(sum(doubles));
```

11. Lower bound

A **lower bound** specifies the minimum/general type that the supplied type must be related to.

Syntax:

? super Type

For example:

List<? super Integer>

means:

A list whose element type is Integer or a supertype of Integer.

Possible types include:

List<Integer>

List<Number>

List<Object>

But not:

List<Double>

List<String>

12. Why does ? super Integer work?

Consider:

List<? super Integer> list;

The actual list could be:

List<Integer>

List<Number>

List<Object>

You can safely add an Integer:

list.add(10);

Why?

Case 1

List<Integer>

An Integer can be added.

Case 2

List<Number>

An Integer is a Number.

Case 3

List<Object>

An Integer is an Object.

Therefore:

```
list.add(10);
```

is always safe.

13. Lower-bound wildcard = good for writing

The famous rule is:

? super T → Consumer → mainly write into it.

Example:

```
static void addNumbers(List<? super Integer> list) {  
    list.add(10);  
    list.add(20);  
    list.add(30);  
}
```

This can accept:

List<Integer>

List<Number>

List<Object>

Example:

```
List<Integer> integers = new ArrayList<>();
```

```
List<Number> numbers = new ArrayList<>();
```

```
List<Object> objects = new ArrayList<>();
```

```
addNumbers(integers);  
addNumbers(numbers);  
addNumbers(objects);
```

All are valid.

14. What can you read from List<? super Integer>?

This is where the difference becomes important.

Consider:

```
List<? super Integer> list;
```

You can add:

```
list.add(100);
```

But when retrieving:

```
Integer value = list.get(0); // ✗
```

is not guaranteed to work.

The compiler only knows that the element type is some supertype of Integer.

It could be:

Integer
Number
Object

Therefore, the only universally safe type to read is:

Object

So:

Object value = list.get(0); // ✓

15. Upper vs lower bounds

This is the central comparison.

Feature	Upper bound	Lower bound
Syntax	? extends T	? super T
Direction	Subtypes	Supertypes
Example	? extends Number	? super Integer
Can represent	Number, Integer, Double...	Integer, Number, Object
Main operation	Reading	Writing
Safe value to read	Upper-bound type	Object
Safe value to add	Generally no specific value	T and its subtypes
Common role	Producer	Consumer

16. Visualizing extends

Suppose:

Object
↑
Number
↑
Integer

For:

List<? extends Number>

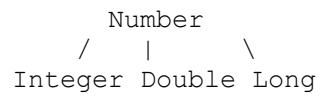
the possible lists are:

List<Number>
List<Integer>
List<Double>
List<Long>

They are all **below Number** in the inheritance hierarchy.

Therefore:

? extends Number



17. Visualizing super

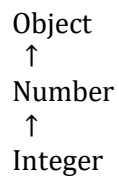
For:

List<? super Integer>

the possibilities are:

List<Integer>
List<Number>
List<Object>

Conceptually:



The possible types are:

Object
Number
Integer

which are Integer itself or its supertypes.

18. PECS rule

A very famous rule for Java generics is:

PECS = Producer Extends, Consumer Super

Producer

If a generic structure **produces values for you to read**:

? extends T

Example:

List<? extends Number>

You consume/read Number values from it.

Consumer

If a generic structure **consumes values that you put into it**:

? super T

Example:

List<? super Integer>

You put Integer values into it.

19. Example combining both

Consider a method that copies elements:

```
static <T> void copy(  
    List<? super T> destination,  
    List<? extends T> source) {  
  
    for (T value : source) {  
        destination.add(value);  
    }  
}
```

This is a classic and very important generic pattern.

Usage:

```
List<Integer> source =  
    Arrays.asList(1, 2, 3);
```

```
List<Number> destination =  
    new ArrayList<>();
```

```
copy(destination, source);
```

Why does this work?

Source

```
List<? extends Integer>
```

The source **produces** Integer values.

Therefore:

```
? extends T
```

Destination

```
List<? super Integer>
```

The destination **consumes** Integer values.

Therefore:

```
? super T
```

20. Collections.copy() uses this principle

The JDK's collection APIs use this exact style of generic design.

Conceptually, the signature is:

```
copy(List<? super T> destination,  
      List<? extends T> source)
```

This allows a source of a subtype and a destination of a supertype.

For example:

Source:

```
List<Integer>
```

Destination:

```
List<Number>
```

works because:

Integer → Number

21. Upper bound in generic methods

Example:

```
static <T extends Number> T maximum(T a, T b) {  
    return a.doubleValue() > b.doubleValue() ? a : b;  
}
```

The compiler knows that T has Number behavior.

But notice that the actual implementation has a subtle issue: comparing values using `doubleValue()` can lose precision for some numeric types. A safer educational example is:

```
static <T extends Number> double add(T a, T b) {  
    return a.doubleValue() + b.doubleValue();  
}
```

Usage:

```
System.out.println(add(10, 20));  
System.out.println(add(10.5, 20.5));
```

22. Lower bounds are primarily wildcard bounds

An important distinction:

You can write:

`<T extends Number>`

but Java does **not** allow:

`<T super Number>` // ❌invalid Java syntax

Lower bounds are expressed with **wildcards**:

`<? super Number>`

This is a crucial rule.

Form	Valid?
<code><T extends Number></code>	✓
<code><T super Number></code>	❌

<? extends Number>	✓
<? super Number>	✓

23. Why doesn't Java have <T super Number>?

Because Java's type parameters are designed around a declared type variable and its upper constraints.

For example:

<T extends Number>

means:

T must be Number or subtype

A lower-bound relationship is represented using a wildcard:

? super Number

meaning:

some unknown type that is Number or a supertype

24. Upper bounds with multiple interfaces

Example:

```
interface Printable {
    void print();
}
```

```
interface Storable {
    void store();
}
```

A type can require both:

```
static <T extends Printable & Storable>
void process(T obj) {

    obj.print();
    obj.store();
}
```

Now T must implement both interfaces.

25. Recursive upper bounds

A more advanced use is:

`<T extends Comparable<T>>`

This means:

T must be comparable with another object of type T.

Example:

```
static <T extends Comparable<T>> T max(T a, T b) {  
    return a.compareTo(b) >= 0 ? a : b;  
}
```

Usage:

```
Integer a = 10;  
Integer b = 20;
```

```
Integer result = max(a, b);
```

Integer implements:

```
Comparable<Integer>
```

so it satisfies:

`<T extends Comparable<T>>`

26. Bounded wildcard vs unbounded wildcard

Unbounded

`List<?>`

means:

A list of some unknown type.

You don't know whether it is:

List<String>
List<Integer>
List<Employee>

Upper bounded

List<? extends Number>

means:

A list of some unknown type that is Number or a subtype.

Lower bounded

List<? super Integer>

means:

A list of some unknown type that is Integer or a supertype.

27. Comparison of all three

Type	Meaning	Example
List<?>	Unknown type	List<String>, List<Integer>
List<? extends Number>	Unknown subtype of Number	List<Integer>
List<? super Integer>	Unknown supertype of Integer	List<Number>

28. A practical example: calculating average

Suppose we want to accept any numeric list:

```
static double average(List<? extends Number> numbers) {  
    double sum = 0;  
  
    for (Number n : numbers) {  
        sum += n.doubleValue();  
    }  
  
    return sum / numbers.size();  
}
```

```
}
```

Now all of these work:

```
List<Integer> integers =  
    Arrays.asList(10, 20, 30);
```

```
List<Double> doubles =  
    Arrays.asList(10.5, 20.5, 30.5);
```

```
System.out.println(average(integers));  
System.out.println(average(doubles));
```

The method is saying:

I don't care which specific Number subtype
you give me.

Just give me something that PRODUCES Numbers.

Therefore:

? extends Number

29. Practical example: adding default values

Suppose:

```
static void addDefaults(List<? super Integer> list) {  
  
    list.add(10);  
    list.add(20);  
    list.add(30);  
}
```

It can accept:

```
List<Integer>  
List<Number>  
List<Object>
```

The method is saying:

I need something that can CONSUME Integers.

Therefore:

? super Integer

30. Generic class vs wildcard bounds

Consider:

```
class Box<T extends Number> {  
}
```

This restricts **what type arguments are allowed for the class**.

```
Box<Integer>    // ✓  
Box<Double>    // ✓  
Box<String>    // ✗
```

But:

```
Box<? extends Number>
```

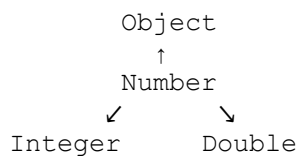
means:

A Box of some unknown type that extends Number.

These are conceptually different.

31. The most important mental model

Think of extends as moving **downward** through the inheritance hierarchy:



? extends Number

You can have:

```
Number  
Integer  
Double
```

? super Integer

You can have:

Integer
Number
Object

Therefore:

? extends T
↓
subtypes of T

while:

? super T
↑
supertypes of T

32. Upper and lower bounds together

The most powerful pattern is:

```
<T>  
List<? extends T> source  
List<? super T> destination
```

For example:

```
static <T> void transfer(  
    List<? extends T> source,  
    List<? super T> destination) {  
  
    for (T item : source) {  
        destination.add(item);  
    }  
}
```

Suppose:

```
List<Integer> source =  
    Arrays.asList(1, 2, 3);  
  
List<Number> destination =  
    new ArrayList<>();  
  
transfer(source, destination);
```

The compiler can infer:

T = Integer

Then:

source
List<? extends Integer>
↓
produces Integer

destination
List<? super Integer>
↓
consumes Integer

This is **PECS in action**.

33. Common mistakes

Mistake 1: Thinking extends means only the exact type

Wrong:

List<? extends Number>

doesn't mean only:

List<Number>

It can also mean:

List<Integer>
List<Double>
List<Long>

Mistake 2: Trying to add to ? extends

List<? extends Number> list;

list.add(10); // ✗

Don't think:

"Integer extends Number, so it should work."

The actual list could be:

List<Double>

Mistake 3: Trying to read as Integer from ? super Integer

```
List<? super Integer> list;
```

```
Integer i = list.get(0); // ✗
```

The actual list could be:

```
List<Number>
```

or:

```
List<Object>
```

So:

```
Object obj = list.get(0); // ✓
```

34. Final cheat sheet

Concept	Syntax	Meaning	Read	Write
Unbounded wildcard	?	Unknown type	Object	Generally no
Upper-bounded wildcard	? extends T	Unknown subtype of T	T	Generally no
Lower-bounded wildcard	? super T	Unknown supertype of T	Object	T
Upper-bounded type parameter	<T extends X>	T must extend/implement X	T	Depends on context
Lower-bounded type parameter	<T super X>	✗ Doesn't exist	—	—

The two rules to memorize

```
? extends T
↓
Producer
↓
READ
```

and

? super T



Consumer



WRITE

Or, in the famous mnemonic:

PECS — Producer Extends, Consumer Super.

The deeper distinction is that **upper bounds constrain a type to the subtype side of a hierarchy**, while **lower-bounded wildcards constrain an unknown type to the supertype side**. This is what allows Java's collection APIs to remain both flexible and type-safe.